

Department of Robotics and Automation

Course Name – Robotics & Automation

Hackathon (Internal Component): Group 12

**Autonomous Agricultural Processing: Automated
Post-Harvest Sorting, Grading and Packaging
Line for Onions**

Project Report

By

Ankur Saxena (24070127019)

Apara Shaligram(24070127023)

Sakshi Goyal (24070127101)

Yajat Alimchandani (24070127137)

Om Sonawane (25070127510)

Raghavendra Pathe (25070127508)



॥वसुधैव कुटुम्बकम्॥

SYMBIOSIS
INSTITUTE OF TECHNOLOGY, PUNE

SYMBIOSIS INSTITUTE OF TECHNOLOGY
A Constituent of Symbiosis International (Deemed University),
Pune-412115
2026-27

Certificate

This is to certify that the project report entitled “Autonomous Agricultural Processing: Automated Post-Harvest Sorting, Grading and Packaging Line for Onions” submitted by

Ankur Saxena (24070127019)

Apara Shaligram(24070127023)

Sakshi Goyal (24070127101)

Yajat Alimchandani (24070127137)

Om Sonawane (25070127510)

Raghavendra Pathe (25070127508)

is a bonafide record of the work carried out by them in the Department of Robotics and Automation, Symbiosis Institute of Technology, Pune, under my supervision during the academic year 2026–27.

Dr. Akshay Saxena

Assistant Professor
Department of Robotics and
Automation

Dr. Arunkumar Bongale

Head of Department
Department of Robotics and
Automation
Symbiosis Institute of
Technology, Pune

Dr. Ramesh BT

Assistant Professor
Department of Robotics and
Automation
Symbiosis Institute of
Technology, Pune

Place:/ Pune

Date: 08/10/2026

Acknowledgement

We would like to thank Dr. Ramesh BT and Dr. Akshay Saxena, Assistant Professors, Department of Robotics and Automation, for guidance and support throughout this project. We also thank the Head of the Department and the faculty of the B.Tech Robotics and Automation Engineering programme at Symbiosis Institute of Technology, Pune, for providing the facilities and software required for the work. Finally, we thank the organisers of the Robotics and Automation Hackathon for giving us this problem statement and the opportunity to work on it.

Group 12

Abstract

This report presents an automated post-harvest processing line for onions, developed for the Group 12 problem statement on autonomous agricultural processing. The line receives onions, classifies each onion by size and quality, sorts it into a grade and prepares it for packaging. An automated feeder releases onions onto a conveyor. A light-beam gauge with four timers determines the size band and a camera checks for sprouting. Each onion is then pushed into one of three grade lanes or carried to a reject bin. A gantry robot packs the graded onions into crates of 10, and a UR10 robot stacks full crates on a pallet of 12.

The control program runs on OpenPLC, the operator interface is built in SCADA-BR and the plant is simulated in CoppeliaSim 4.10. A Python script connects the simulation with the PLC, and all components exchange data over OPC UA through 42 variables hosted on the OpenPLC server. Eight fault codes cover emergency stop, jams, pusher failures, queue overflow, a full reject bin and unexpected onions.

In a 46.6 s reference run the line processed 19 onions (Grade A 5, Grade B 7, Grade C 4, rejected 3) with 100% classification accuracy and a peak processing rate of 42 onions per minute. With OpenPLC in control, all 42 OPC UA variables were verified and the plant sorted onions with 100% PLC-reported accuracy.

Keywords: post-harvest processing, onion grading, PLC, OpenPLC, SCADA-BR, OPC UA, CoppeliaSim, robotic palletizing

Contents

1 Project purpose and requirements	6
2 System architecture and simulation	7
3 Classification and grading	8
4 Automated feeding and product tracking	9
5 GX Works3 conveyor and actuator logic	10
6 OpenPLC implementation and GX Works3 logic	11
7 Structured Text and PLC communication	12
8 Sorting and preparation for packaging	13
9 Robot motion and RT ToolBox	14
10 Fault and rejection handling	15
11 Quantity monitoring and performance indicators	16
12 Production trends and classification quality	17
13 Packing status results and discussion	18
14 Validation limitations and future work	19
15 Conclusion and references	20

Implementation versions

The project develops the complete onion processing flow in simulation. The earlier design uses grade lanes, a gantry packing stage, ten onions per crate and a UR10 palletizer. The supplied Python implementation uses pushers that fill grade crates directly, six cells per crate and a single palletizing robot. Both versions use three accepted grades and a separate reject outcome.

The GX Works3 ladder sequence was translated into Structured Text and used in OpenPLC Runtime to control the simulated line. The report presents the ladder design, its OpenPLC implementation and the RT ToolBox handling demonstration. OpenPLC runs are distinguished from the Python built-in controller runs shown in the new dashboard screenshots.

1 Project purpose and requirements

Our project addresses the Group 12 Autonomous Agricultural Processing problem by designing an automated post-harvest line for onions. Each onion enters through a feeder, receives a size and quality classification, is directed to its grade destination and is collected in a crate for palletizing. Monitoring records how many products receive each grade and how consistently the classification matches the reference.

Problem and objectives

Manual grading depends on the judgement of the operator and becomes difficult to keep consistent as the quantity increases. The project therefore separates sensing, decision making and actuation. A repeatable measurement supports the decision, the PLC applies the same grading rules to every item, and the sorting mechanism routes products without requiring an operator to handle each onion.

Required feature	Project implementation
At least three categories	Grade A, Grade B and Grade C, plus reject
Automated feeding	Timed hopper gate releases individual onions onto the belt
Classification decision	Diameter thresholds with a sprout rejection override
Sorting	Three controlled pushers separate accepted grades
Packaging	Grade crates are filled and full crates are palletized
Fault and rejection handling	Product reject path; emergency stop, jam and link supervision according to controller mode
Quantity monitoring	Grade totals, rejected quantity, rate and classification accuracy

Scope of the system

The input is harvested onion product ready for grading. Washing, drying, peeling and processing inside the bulb are outside the scope. Size is represented by diameter; sprouting is the quality defect modelled. The system prepares bulk crates for dispatch rather than performing bag sealing, weighing to a sales specification or applying labels.

The delivered work is a simulation and software control demonstration. Simulated objects, sensor events and robot motions allow the process sequence to be evaluated. A physical installation would additionally require sensor calibration, mechanical design, wiring and commissioning. The grading limits are project choices and should be agreed with the user of a real packing line.

2 System architecture and simulation

The control sequence was developed as GX Works3 ladder logic, then implemented in Structured Text in OpenPLC. OpenPLC Runtime executes the control program, while Python exchanges plant measurements and output commands through OPC UA. Python also builds and updates the CoppeliaSim scene. The supplied script retains a built-in ladder mode for offline verification and demonstration.

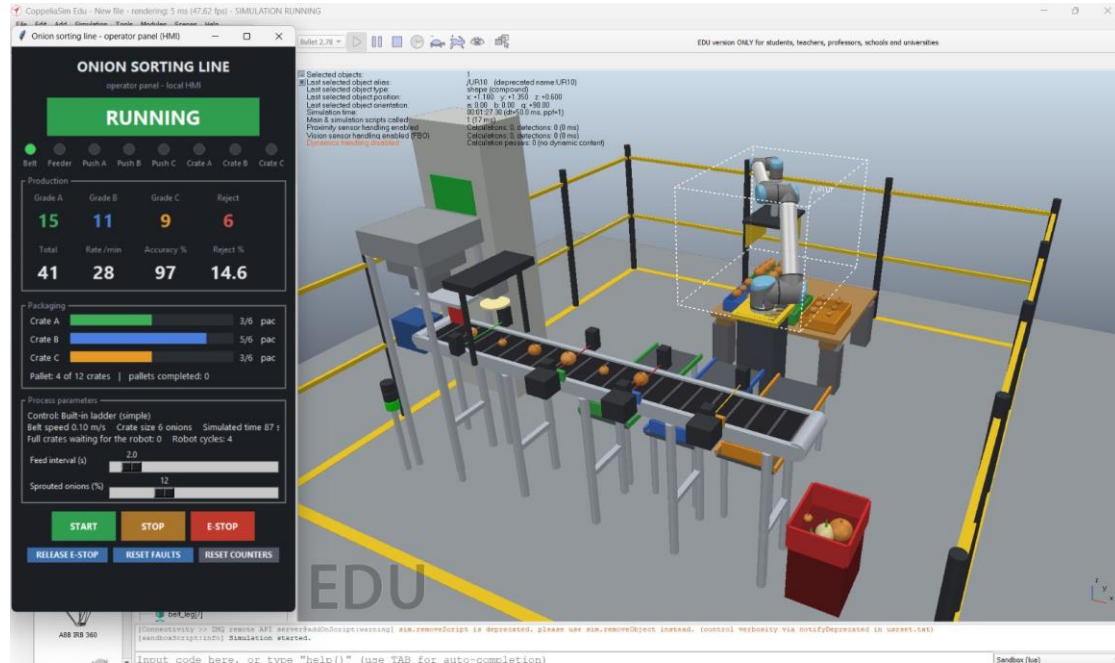


Figure 1 Supplied CoppeliaSim scene with the running local operator HMI

Component	Responsibility
CoppeliaSim	Displays the conveyor, grading stations, crates, robot and pallet
Python plant model	Creates onion data and sensor events; moves products and crates
Built-in PLC interpreter	Executes the selected simple or full ladder program
OpenPLC Structured Text	Executes the ST implementation of the GX ladder; supplies actuator commands and KPIs through OPC UA
Operator monitoring	Local HMI and browser dashboard; SCADA-BR is described in the earlier design

The script connects to CoppeliaSim through the ZeroMQ remote API. This API supports control from an external program, including synchronized stepping [6]. In this project, shapes are moved kinematically with physics disabled. Sorting results therefore reflect the programmed sensor and actuator sequence, while contact forces, bruising and uncontrolled rolling are not represented.

The scene file onion_sim.ttt is the saved project scene. The CoppeliaSim recording accompanies the simulation demonstration, while the GX Works3 recording accompanies the PLC demonstration. These artifacts serve different roles from the source code: the code defines the logic and the images illustrate the implementation.

3 Classification and grading

Every onion receives one final grade. Size determines the accepted band only when the quality check is clear. A sprout detection overrides an otherwise acceptable size and directs the onion to rejection. The same thresholds appear in the supplied Structured Text and in the Python reference grading function.

Outcome	Diameter condition	Quality condition	Code
Grade A	70 mm to below 90 mm	No detected sprout	1
Grade B	55 mm to below 70 mm	No detected sprout	2
Grade C	40 mm to below 55 mm	No detected sprout	3
Reject	Below 40 mm or at least 90 mm	Any detected sprout also rejects	4

Size measurement principle

For the built-in belt model, the onion interrupts a beam while moving past the size gauge. With constant belt speed v and beam interruption time t , the measured travel dimension is $d = v \times t$. At 100 mm/s a 10 ms timer tick represents 1 mm. For example, a 0.62 s interruption corresponds to 62 mm and gives Grade B when no sprout is detected.

The earlier full ladder uses multiple threshold timers. The current simple ladder reads elapsed timer T20 at the falling edge of X5, stores the measurement in D11 and applies thresholds in sequence. Higher thresholds overwrite the provisional lower grade, with the final 90 mm check assigning reject. Rung order is therefore part of the classification logic.

Quality decision and boundaries

The simulated sprout state is evaluated with a configurable detection probability, set to 0.96 by default. This is a model parameter rather than measured camera performance. The Python reference function below supplies the true grade for the classification audit.

Code excerpt 1 Python reference grading from onion_sorting_line.py

```
def true_grade(d, sprouted):
    if sprouted or d < 40 or d >= 90:
        return 4
    if d >= 70:
        return 1
    if d >= 55:
        return 2
    return 3
```

An unsprouted onion at exactly 40, 55 and 70 mm belongs to C, B and A respectively; at 90 mm it is rejected. A detected sprout overrides every size band. The following Structured Text excerpt applies the same grading thresholds.

```
IF SPROUTED OR DIAMETER_MM < 40.0 OR DIAMETER_MM >= 90.0 THEN
    Grade := 4;
ELSIF DIAMETER_MM >= 70.0 THEN Grade := 1;
ELSIF DIAMETER_MM >= 55.0 THEN Grade := 2;
ELSE Grade := 3;
END_IF;
```

4 Automated feeding and product tracking

Automated feeding gives each onion enough separation to be measured and sorted independently. In the supplied runner, the default feed interval is 2.0 s. The Structured Text feeder clamps the interval to 15 through 100 tenths of a second, equivalent to 1.5 through 10.0 s, and opens the gate for 0.5 s.

Spacing and nominal rate

At 100 mm/s and a 2 s interval, the nominal spacing between release events is 200 mm. The corresponding theoretical input rate is $60 / 2 = 30$ onions/min. A 90 mm product occupies the beam for about 0.9 s, leaving time before the next nominal arrival. Reducing the interval increases input rate but also increases the number of products simultaneously being tracked.

The feed rate is a set point rather than a guarantee of packaged output. Start-up travel, stopped time, quality rejection, crate replacement and the robot cycle all affect the final useful rate. A stable feed interval is valuable because it makes timing and queue occupancy predictable during a demonstration.

Tracking in the built-in ladder

After the camera exit, the simple ladder stores the final grade in a sixteen-entry ring buffer D100 through D115. D20 is the write pointer. D21, D22 and D23 are read pointers for stations A, B and C. Each pointer wraps by a word AND with 15, keeping its value within 0 through 15.

Station A sees all products. Station B skips entries already removed as Grade A, and station C skips entries removed as Grade A or B. This keeps later stations aligned with the products still on the belt. A grade is retained until the associated product arrives, rather than being inferred from a fixed travel delay alone.

Tracking in Structured Text

The external controller uses CAM_ID to detect a new measurement and stores Route[CAM_ID MOD 16]. PE_A_ID, PE_B_ID and PE_C_ID identify the product at each pusher. PUSHED_A_ID, PUSHED_B_ID and PUSHED_C_ID acknowledge a transfer. Repeated observations of the same product must not create a second push or a second classification count.

The sixteen-slot route array must not be overwritten while an older product still needs its grade. A faster or longer line needs ownership checks or more queue capacity. The controller independently calculates a decision accuracy KPI using the following excerpt.

Code excerpt 2 Classification accuracy from OnionSorting_body.st

```
IF TESTED > 0 THEN
    ACCURACY_PCT := DINT_TO_INT((Correct * 100) / TESTED);
END_IF;
```

5 GX Works3 conveyor and actuator logic

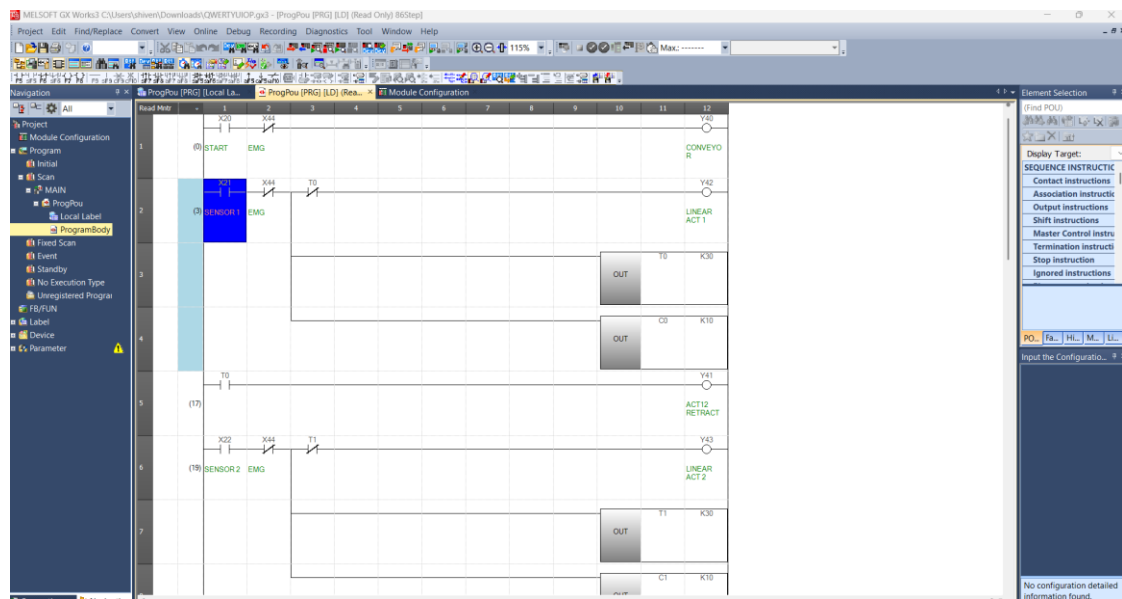


Figure 2 Supplied GX Works3 ladder screenshot showing rungs 1 through 7

The GX Works3 screenshot shows a compact sensor and actuator demonstration. Rung 1 places START input X20 in series with the normally closed EMG contact X44 and drives conveyor output Y40. In the visible logic, the conveyor follows this permissive combination. A separate run latch is not visible in this rung.

Rung 2 uses sensor X21, the X44 emergency interlock and a normally closed timer contact T0 to energize Y42, labelled LINEAR ACT 1. Parallel branches start T0 with preset K30 and counter C0 with preset K10. Rung 5 uses the completed T0 contact to drive Y41, labelled ACT12 RETRACT. The labels are reproduced as shown rather than interpreted as a full actuator wiring diagram.

Rungs 6 and 7 repeat the pattern for sensor X22, actuator output Y43 and timer T1 with K30. This repeated structure gives each grading station its own actuation sequence. The screenshot does not show the complete third-station sequence, all reset conditions or actuator position feedback.

Visible device	Meaning in this screenshot
X20 / X44	Start input / emergency interlock
Y40	Conveyor command
X21 / X22	Station sensor inputs
Y42 / Y43	Actuator commands for stations 1 and 2
T0 / T1 with K30	Actuation timing presets
C0 with K10	Counter threshold for a ten-item batch

K30 is a numerical timer preset. Its duration depends on the configured PLC timer time base, which is not established by the image alone. The GX addresses are specific to this demonstration and must not be substituted directly for the Python X0, Y0 and related addresses. A hardware implementation needs a verified I/O mapping and exclusive extend and retract commands.

6 OpenPLC implementation and GX Works3 logic

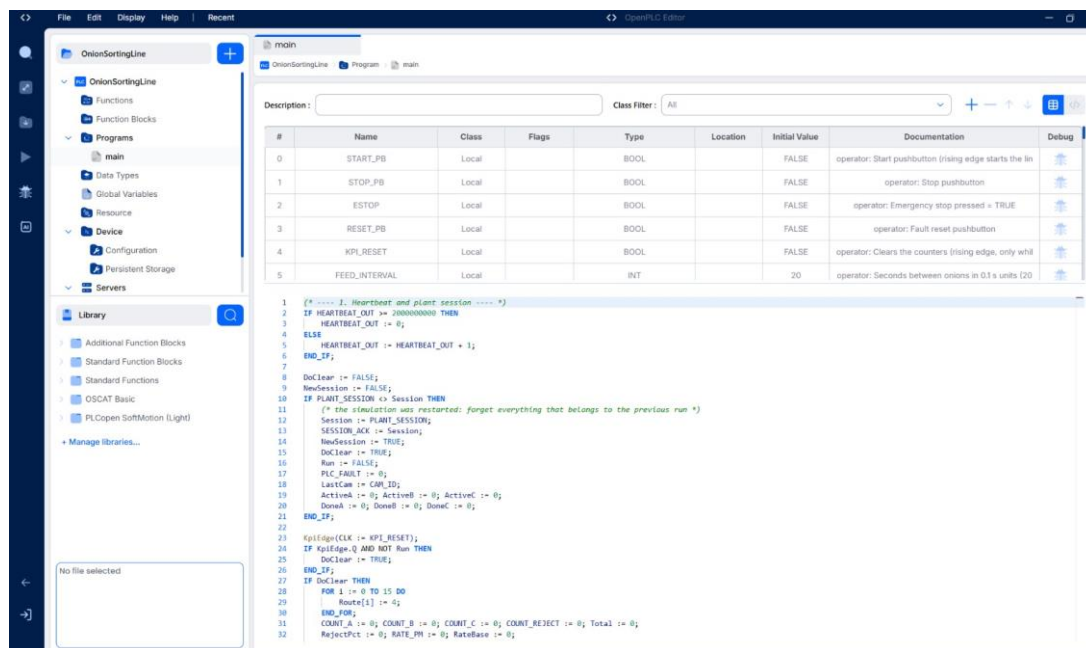


Figure 10 OpenPLC Editor showing the declared variables and Structured Text program

OpenPLC Editor shows the OnionSortingLine project, operator-variable declarations and Structured Text heartbeat, session-reset and KPI-reset logic. The GX Works3 ladder was implemented in ST and used in OpenPLC Runtime for control. Chapter 7 explains the controller and communication path.

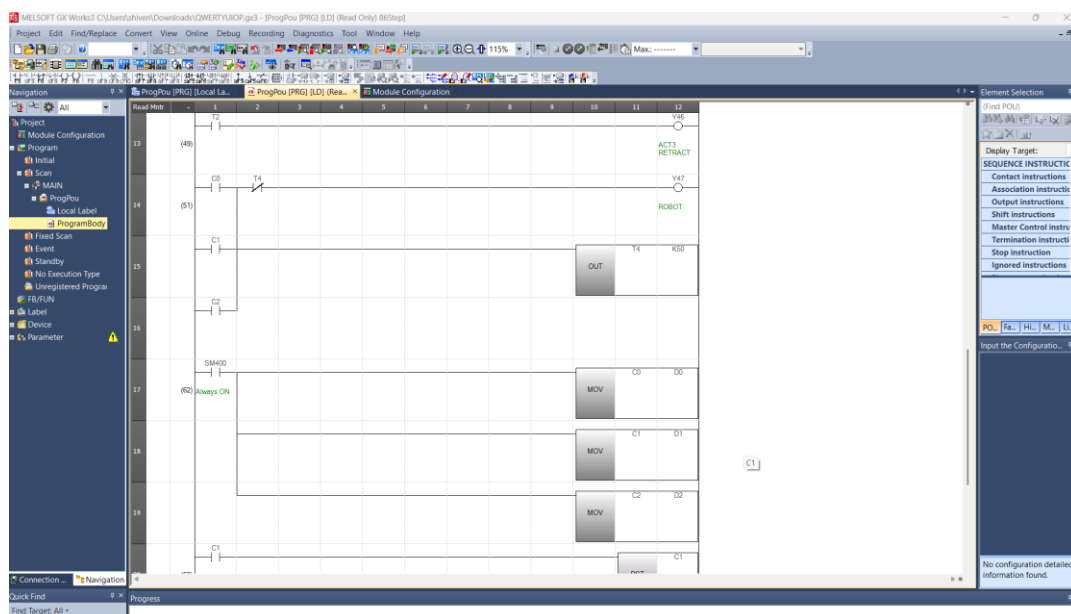


Figure 3 Supplied GX Works3 ladder screenshot showing rungs 13 through 19

In GX Works3, T2 drives third-actuator retraction Y46. C0 and the T4 interlock drive robot output Y47; C1 or C2 starts T4 with K50. Rungs 17–19 copy counters C0, C1 and C2 to D0, D1 and D2 under SM400 for quantity monitoring.

Robot requests still require a completion handshake: the controller should confirm that a crate is handled before resetting its batch counter.

7 Structured Text and PLC communication

The GX Works3 ladder was implemented in Structured Text as OnionSorting.st and used in OpenPLC for control. OpenPLC Editor builds and uploads the program; Runtime scans it every 50 ms [8]. Its 42 tags cover commands, measurements and outputs. The newer dashboard screenshots use built-in Python control.

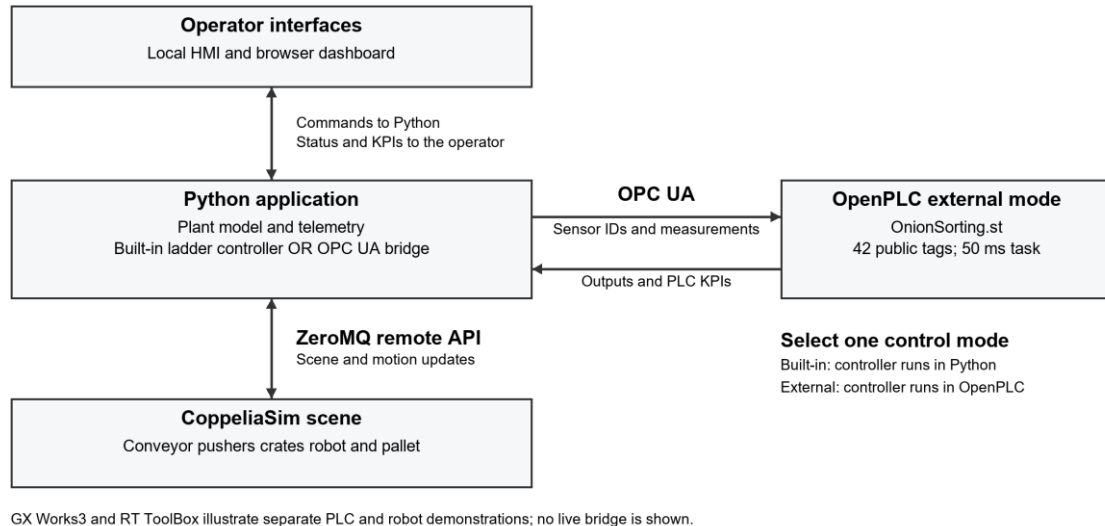


Figure 9 Communication workflow for built-in and external controller modes

Scan sequence

OpenPLC evaluates faults and run state, then executes feeding, classification, KPIs, pushers and robot requests. Contacts and coils become Boolean conditions and assignments; timers use TON. A new CAM_ID triggers one classification, checked against TRUE_GRADE.

Declare variables, compile and upload the ST program, expose 42 tags and run --opc-check. Select --plc opcua for OpenPLC control. PLANT_SESSION clears stale data; SESSION_ACK confirms the reset.

Communication and timing

Python writes measurements and sensor IDs to OpenPLC through OPC UA and reads actuator commands and KPIs [7]. OpenPLC decides; Python applies commands. CAM_ID is written after its associated measurements.

Python controls the scene through Coppeliasim's ZeroMQ remote API [6]. The exact connection helper below retrieves the sim API used by the scene builder and motion updates.

Code excerpt 3 Coppeliasim connection from onion_sorting_line.py

```
def get_sim(host, port):
    from coppeliasim_zmqremoteapi_client import RemoteAPIClient
    client = RemoteAPIClient(host, port)
    sim = client.require('sim')
    return client, sim
```

OpenPLC detects plant stalls after one second and link loss after ten seconds. Earlier tests found 42/42 tags and demonstrated OpenPLC sorting, with longer-run interruptions. SCADA-BR belongs to the earlier supervisory design.

8 Sorting and preparation for packaging

Three pushers route accepted onions into their matching grade destinations. A Grade A onion leaves the conveyor at station A; Grade B and C continue to their respective stations. Rejects match no accepted grade and proceed to the reject bin at the belt end. Quality rejection therefore remains a normal product outcome while the line continues operating.

Pusher control and acknowledgements

The simple ladder holds each pusher for nominally 0.3 s. Structured Text tracks an Active product ID until the matching pushed-ID acknowledgement, a four-second timeout or the next nonzero product ID.

DoneA, DoneB and DoneC prevent a repeated push for one product. A timed-out command clears without a dedicated pusher fault; physical actuation therefore needs verified transfer feedback.

Crate filling and palletizing

The Python plant uses six-cell crates. Each pusher sends an onion into a reserved cell. A full crate rolls to the pick-up position, a replacement arrives and the single palletizer moves the full crate to the pallet.

Packaging parameter	Earlier design	Supplied Python implementation
Onions per crate	10	6 by default; six physical cells
Transfer into crates	Gantry picks from grade lanes	Pushers fill grade crates directly
Pallet capacity	12 crates	12 crates
Pallet arrangement	3 layers of 4	2 layers of 6
Robot task	Gantry filling and UR10 palletizing	Single palletizing robot

The Structured Text robot calls depend on crate readiness, pallet space and emergency-stop status. A call is a request rather than confirmation that a crate is palletized. The following excerpt also shows the stopped-state conveyor and feeder commands.

Code excerpt 4 Robot requests and stopped state from OnionSorting_body.st

```
CALL_A := CRATE_A_READY AND NOT PALLET_FULL AND NOT ESTOP;  
CALL_B := CRATE_B_READY AND NOT PALLET_FULL AND NOT ESTOP;  
CALL_C := CRATE_C_READY AND NOT PALLET_FULL AND NOT ESTOP;
```

```
IF NOT Run THEN  
    FEED_GATE := FALSE;  
    CONVEYOR_RUN := FALSE;  
    PUSH_A := FALSE; PUSH_B := FALSE; PUSH_C := FALSE;  
END_IF;
```

Crates provide bulk packing. Bag closure, net-weight checks and labels remain future additions. Robot calls are not gated by Run; a separate stop-policy interlock is needed if the robot must stop on every line stop.

9 Robot motion and RT ToolBox

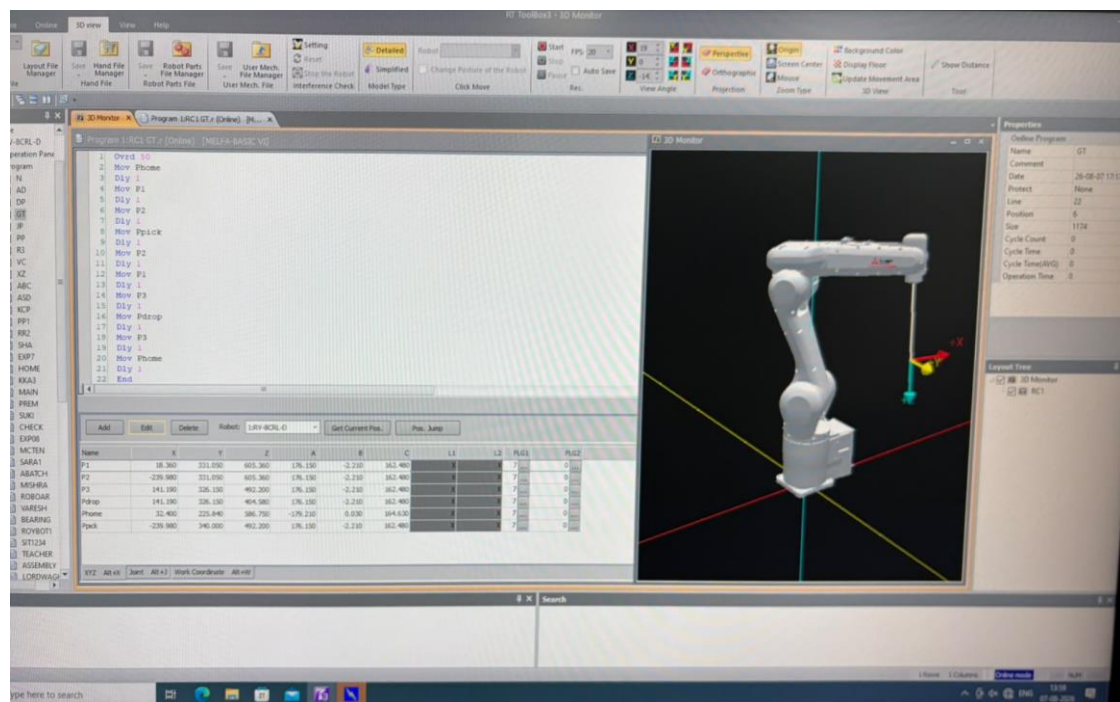


Figure 4 Supplied RT ToolBox robot view and taught point sequence

The RT ToolBox image shows a Mitsubishi robot in a 3D monitor and a MELFA-BASIC VI point sequence. The visible program sets OvrD 50 and uses Mov commands with Dly 1 statements. Its taught position list contains P1, P2, P3, Pdrop, Phome and Ppick.

Phome → P1 → P2 → Ppick → P2 → P1 → P3
 → Pdrop → P3 → Phome

This sequence illustrates an approach, pick position, retreat, transfer, drop position and return home. The intermediate points provide clearance between the robot and the working area. The one-second dwells visibly separate the movement stages, but the screenshot does not establish actual travel time, grip force or production capacity.

Relation to the CoppeliaSim robot

The supplied Python implementation uses a UR10 library model by default for palletizing. The script constructs its tool and joint geometry and drives the joints with a numerical inverse kinematics solver. The Mitsubishi demonstration and the UR10 scene are separate robot implementations of the handling concept; a taught point from one cannot be copied into the other without coordinate and tooling conversion.

The original report documents a UR10 test with 30 visited targets and a maximum position error of 0.74 mm. These are earlier simulated kinematic results, not measured physical robot accuracy. A reachability check should cover every crate pick position, approach position and pallet layer, and should be repeated after modifying the base position, tool offset or pallet layout.

Before physical handling, the robot interface should include ready, busy, request accepted, completed and fault states. A grip confirmation should establish that the crate is retained before transfer. The motion path must also provide clearance from the conveyor, full crates and the top pallet layer. A sequence of point moves alone does not verify those conditions.

10 Fault and rejection handling

Product rejection and machine faults require different responses. An onion rejected for size or sprouting is removed from accepted output and recorded as reject. A jam, emergency stop or lost plant link stops the affected process. This distinction allows normal quality variation to be handled without hiding a malfunction.

Condition	Implemented response	Controller scope
Sprout or unacceptable size	Assign reject grade 4 and bypass accepted pushers	All grading implementations
Emergency stop	Clear run; conveyor, feeder and pushers stop	Simple ladder and Structured Text
Beam blocked for 5 s while running	Jam condition stops run; reset is available	Simple ladder and Structured Text
Heartbeat unchanged for 10 s	Set PLC_FAULT = 4 and stop run	Structured Text
Pusher active for 4 s without transfer	End the pusher command; no dedicated fault is raised	Structured Text
Queue, transfer or reject-bin faults	Additional fault logic is available in the full ladder	Full ladder option

Recovery sequence

The operator first identifies and clears the cause, then resets the relevant condition and deliberately restarts the line. The Structured Text emergency fault clears after ESTOP is released, but Run remains false until a new Start edge. Its link fault clears when communication returns, also requiring Start again. A jam fault uses the reset path.

Recovery should retain or reconcile products already on the belt. Resetting all counters or routing entries while products remain in flight can detach a product from its grade. The simulation provides a session reset for a fresh run; an operating line needs a deliberate drain or recovery procedure when restarting after a fault.

Interlocks to complete before hardware use

The Structured Text stopped-state block disables feeding, the conveyor and pushers, but CALL_A, CALL_B and CALL_C are gated by crate readiness, pallet availability and ESTOP rather than by Run. A normal Stop or another line fault therefore does not necessarily withdraw a robot call. Whether the robot may finish an isolated cycle must be specified and verified before connecting actual equipment.

The full-ladder fault coverage is broader than the default simple mode. A test report should state which mode was used rather than attributing all eight earlier fault codes to every version. The simulated emergency command also needs to be distinguished from the separate safety controls and physical guarding required for real machinery.

11 Quantity monitoring and performance indicators

The live dashboard reports classification, throughput, grade quantities, equipment effectiveness and packing progress. The new screenshots show the built-in simple ladder in operation. At simulated time 5 min 50 s, the summary records 79 classified onions, with 81 fed and two still on the line. The following values belong to this specific snapshot.

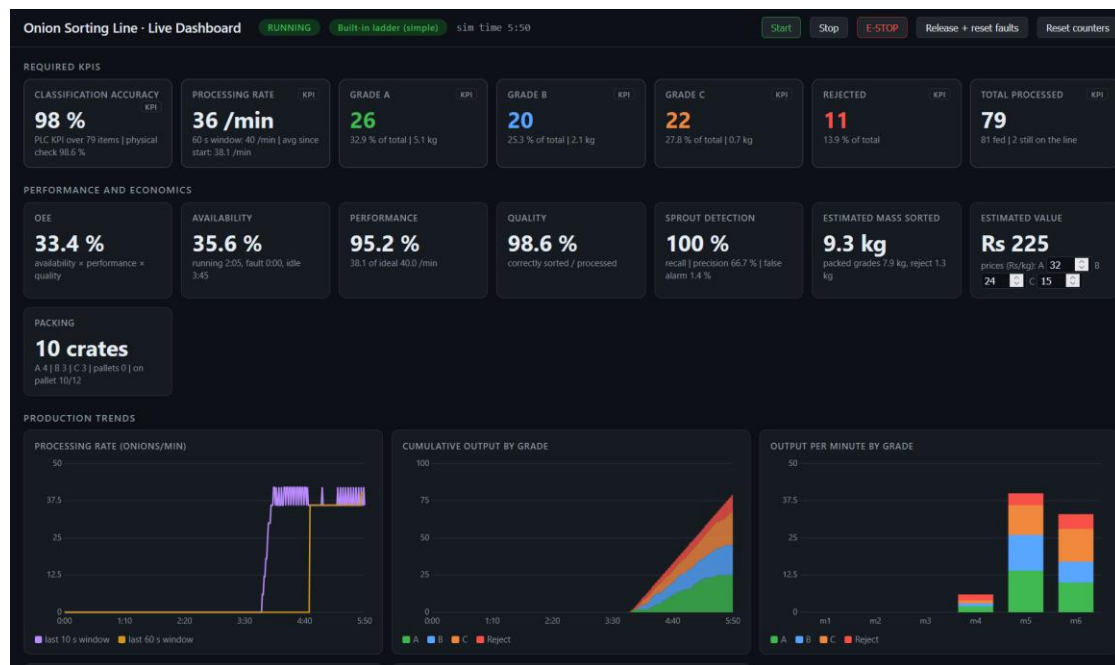


Figure 5 Live dashboard showing required KPIs performance economics and packing

KPI	Displayed value	Interpretation
Classification accuracy	98%; 79 items	PLC decision KPI; physical check 98.6%
Processing rate	36 onions/min	Displayed PLC rate; 60 s window 40/min; running average 38.1/min
Grade-wise quantity	A 26; B 20; C 22; reject 11	Total 79; accepted quantity 68
Overall equipment effectiveness	33.4%	Availability 35.6% × performance 95.2% × quality 98.6%
Sprout detection	Recall 100%; precision 66.7%	False alarm 1.4% in the displayed sample
Mass and value estimates	9.3 kg; Rs 225	Accepted-grade mass 7.9 kg; reject mass 1.3 kg; rounded model estimates
Packing	10 crates; pallet 10/12	Completed crates A 4, B 3, C 3; no completed full pallet

Grade shares are A 32.9%, B 25.3%, C 27.8% and reject 13.9%. The reject share is $11 / 79 \times 100 = 13.92\%$. The three rate measures use different update rules and windows; the displayed 36/min should not be replaced by the separate 40/min rolling-window value. Quantity counters describe classification events, while packing counts describe completed crate transfers.

12 Production trends and classification quality



Figure 6 Rate grade output accuracy and reject-share trends

Production is flat during the initial idle period and rises after feeding starts. The 10 s rate changes more rapidly than the 60 s rate because fewer products determine each short window. The cumulative grade chart shows A, B and C output growing together; the rejected share settles near the low teens. These trends explain stable active throughput alongside low availability over the complete elapsed session.

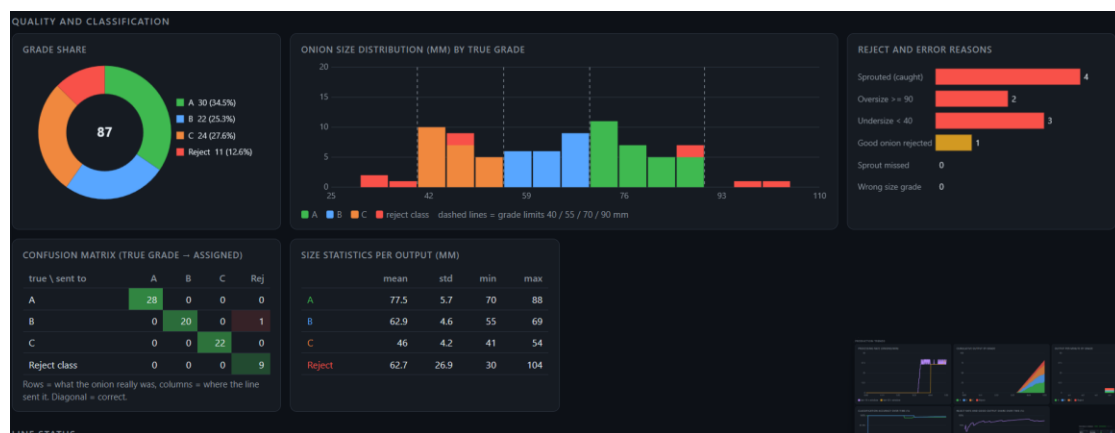


Figure 7 Grade shares size distribution rejection reasons and confusion matrix

The later grade-share widget records 87 products: A 30 (34.5%), B 22 (25.3%), C 24 (27.6%) and reject 11 (12.6%). This differs from the earlier 79-product snapshot because the line continued operating. Mean output diameters are A 77.5 mm, B 62.9 mm and C 46.0 mm, with respective ranges 70–88, 55–69 and 41–54 mm. These values align with the configured size bands.

The displayed physical confusion matrix contains 80 routed items: 28 true A sent to A, 20 true B sent to B, one true B sent to reject, 22 true C sent to C and nine true rejects sent to reject. Its diagonal gives $79 / 80 = 98.75\%$ physical correctness for that matrix sample. The rejection-reason panel lists four caught sprouts, two oversize, three undersize and one good onion rejected, with zero missed sprouts and zero wrong-size grades.

The grade-share total, confusion-matrix total and reason counts are different samples or update stages. They must not be combined into one denominator or described as a single synchronized audit. Sprout recall of 100% means no labelled sprouts were missed in the displayed sample; precision of 66.7% also shows false-positive sprout decisions. Neither percentage establishes real-camera performance.

13 Packing status results and discussion



Figure 8 Crate filling pallet status time breakdown and simulation test controls

The later line-status image shows ten robot palletizing cycles and 10 of 12 pallet positions occupied. At six onions per completed crate, 60 onions are stacked. Crates A and B display 6/6 and ready; C displays 7/6. The latter is an over-target counter indication that should be reconciled with physical crate occupancy, rather than interpreted as a validated seven-cell crate. No active fault and no fault events are shown.

Evidence	Key result	Operating context
New dashboard summary	79 classified; 98% PLC accuracy; 36/min displayed	Built-in simple ladder; snapshot at 5:50
Later quality screenshot	87 in grade-share widget; physical matrix 79 correct of 80	Later snapshot; different counting stages
Later line status	10 crates; 60 onions stacked; 0 fault events	Feed interval shown as 1.5 s; pallet capacity 12
Repeatable offline verification	58 classified: A 23, B 14, C 15, reject 6; 98% KPI	120 s; seed 1; 2 s feeding; six-onion crates
Earlier report reference	19 grade totals; reported 100% decision accuracy	46.6 s earlier demonstration; peak 42/min over 10 s

At 1.5 s feeding the theoretical input rate is 40 onions/min. The dashboard performance of 95.2% follows approximately $38.1 / 40 \times 100$. Availability is 35.6% in the summary, with 2:05 running and 3:45 idle; the later status image shows 38.8% running and 61.2% idle. OEE of 33.4% is consistent with $0.356 \times 0.952 \times 0.986$. Initial stopped time, rather than recorded faults, accounts for most lost availability in these images.

The later hourly projections are 2,298 onions and 275.8 kg. They extrapolate current model performance; they are not measured one-hour production. Value uses configurable model prices of Rs 32, 24 and 15 per kg for A, B and C. The Coppeliasim HMI image is another snapshot: 41 products (A 15, B 11, C 9, reject 6), 28/min, 97% accuracy and four palletized crates. Its 2 s feed setting must be distinguished from the later 1.5 s dashboard setting.

14 Validation limitations and future work

Validation should cover each stage of the flow as well as the transitions between stages. The requirement matrix is satisfied at the design and simulation level, while deployment readiness requires additional tests. The checks below separate reviewed or executed evidence from tests still needed for acceptance.

Check	Evidence or acceptance criterion	Status
Three grades and reject	Same 40, 55, 70 and 90 mm boundaries in Python and ST	Code reviewed
Automated feed and production	120 s offline run produces 58 classifications	Executed
Quantity reconciliation	A + B + C + reject equals 58	Verified
PLC demonstration	Visible GX start, interlock, actuator and counter logic	Images reviewed
Robot handling concept	Visible RT ToolBox taught pick and drop sequence	Image reviewed
Threshold boundary tests	Check each exact threshold and sprouted override	Required test
Fault recovery	Inject E-stop, jam and heartbeat loss; confirm deliberate restart	Required test
Sorting confirmation	Compare each classified ID with its actual destination	Required test
External controller endurance	Sustained OPC UA run with no missed IDs or unexplained stop	Required test

Limitations

The sensor model supplies diameter and sprout information rather than analysing real camera images. Disabled physics omits impacts, rolling, belt slip and damage. The offline check does not exercise CoppeliaSim rendering, network timing or OpenPLC task scheduling. The supplied GX images show selected rungs rather than the entire program, and the RT ToolBox image shows motion logic without a complete gripper handshake.

Development priorities

First, establish one release configuration with an agreed crate capacity, pallet pattern, I/O list and fault set. Then test grade boundaries, products in flight during stop and restart, queue reuse and transfer acknowledgements. Store separate counters for classified, accepted, physically sorted and palletized product so failures can be located rather than hidden in an aggregate total.

Next, repeat external controller endurance tests while recording scan period, heartbeat age and communication latency. Add actuator confirmation, full-crate and pallet interlocks, and a defined response to timeouts. Finally, calibrate physical diameter and quality sensing and evaluate gentle handling, crate retention, guarding and the emergency-stop system. Weight-based packing and lot traceability can follow once the basic transfer sequence is reliable.

15 Conclusion and references

Group 12 develops a post-harvest onion processing system that follows Product Input → Classification → Grading → Sorting → Packaging. Timed feeding provides separated products, size thresholds and sprout detection produce a classification decision, three pushers separate accepted grades, and grade crates are prepared for palletizing. Product rejects and machine faults are handled as different conditions.

The software demonstrates the required three accepted grades and monitors grade quantities, classification accuracy and processing rate. The supplied GX Works3 screenshots support the conveyor, actuator, batch counter and robot-command concepts. The RT ToolBox image supports the approach, pick, transfer and drop sequence, while CoppeliaSim provides a visual processing cell and the Python model supports repeatable software verification.

The GX Works3 control sequence was implemented in Structured Text and executed in OpenPLC, with Python linking the controller to CoppeliaSim through OPC UA. Earlier OpenPLC runs verify external control; the newer dashboard and 120 s offline verification use the built-in Python ladder. The offline run classified 58 onions at 98% reported accuracy, while the new dashboard separately records 79 classifications and ten completed crates. These results must remain associated with their controller mode and operating settings.

Project sources

[1] Group 12, Group12_Autonomous_Agricultural_Processing_Report.docx. Original project report and embedded simulation, dashboard and terminal figures.

[2] onion_sorting_line.py. Supplied controller, plant model, robot motion, monitoring and runner source. Offline verification: --offline --duration 120 --no-dashboard --no-hmi --seed 1.

[3] OnionSorting.st and OnionSorting_body.st. Supplied Structured Text controller and program body.

[4] Supplied GX Works3 and RT ToolBox screenshots; dashboard screenshots 220930, 220939, 220951 and 221002; CoppeliaSim Scene with HMI.jpeg. PLC robot and performance evidence. OpenPLC Screenshot.jpeg shows the implemented ST program in the editor.

[5] onion_sim.ttt; CoppeliaSim Recording.mp4; GxWorks3 Recording.mp4. Saved scene and accompanying demonstration recordings.

Technical references

[6] Coppelia Robotics, ZeroMQ remote API documentation. <https://manual.coppeliarobotics.com/en/zmqRemoteApiOverview.htm>. Accessed 7 October 2026.

[7] OPC Foundation, Unified Architecture. <https://opcfoundation.org/about/opc-technologies/opc-ua/>. Accessed 7 October 2026.

[8] Autonomy Logic, OpenPLC Runtime v4 README and Editor Integration. <https://github.com/Autonomy-Logic/openplc-runtime/blob/main/README.md> and https://github.com/Autonomy-Logic/openplc-runtime/blob/main/docs/EDITOR_INTEGRATION.md. Accessed 7 October 2026.